

Authentication (OAuth2)

To access Upwork API, you need to go through an authentication process. Currently, we support the OAuth 2.0 method.

i You need to authenticate for all requests following the OAuth 2.0, RFC 6749.

OAuth2 Basics

The OAuth 2.0 protocol enables websites or applications (clients) to access protected resources from a Web service (server) via an API, without requiring resource owners to disclose their service provider credentials to the clients. For more information on the OAuth 2.0 workflow process, visit the OAuth 2.0 protocol [official site](#).

Client ID

For each application you develop, you need to obtain new client credentials. These include a client identifier and a client shared-secret. You can request these credentials in the [API Center](#) while logged into your Upwork account (select Key Type OAuth 2.0). You will receive a key (client_id) for each client identifier and client shared-secret you request.

Required X-Upwork-API-TenantId header

Each request requires the organization context. In order to specify on behalf of which context the request should be executed, you need to send X-Upwork-API-TenantId header. When the header is missing, the request will use the default organization of the user.

How X-Upwork-API-TenantId header can be used to set the Organization for the execution context

In order to get a value for the X-Upwork-API-TenantId header, you need company selector query.

Sample company selector query:

```
query {
  companySelector {
    items {
      title
      organizationId
    }
  }
}
```

The request with X-Upwork-API-TenantId would look like:

```
curl --request POST \
  --url https://api.upwork.com/graphql \
  --header 'Authorization: bearer oauth2v2_f5*****' \
  --header 'Content-Type: application/json' \
  --header 'X-Upwork-API-TenantId: 470*****' \
  --data '{"query":"query { \
    organization { \
      id \
      childOrganizations { \
        id \
        name \
      } \
    } \
  }"}'
```

In this case query.organization and organization.childOrganizations will contain information about organization with id 470*****97.

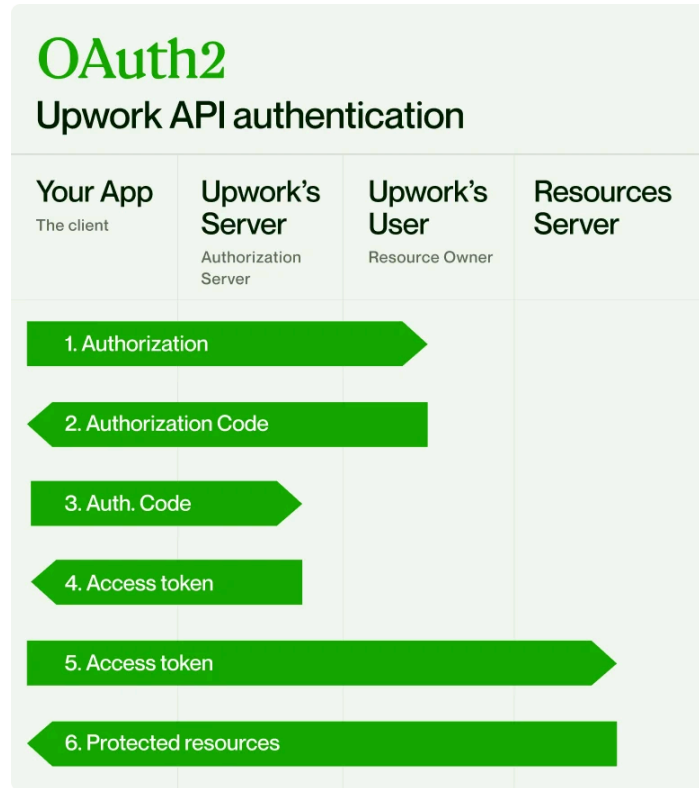
Without the header query.organization will contain the information about 584*****76 which is the default organization for the user.

How the default Organization is selected when the header is not present

A default organization is one of the organizations that the user has access to. It is the same Organization that gets selected as the current Organization when a user logs in into the platform.

Flow and Supported Grants

1. Authorize the application using one of the available grants and obtain an Authorization code in Code Authorization Grant
2. Get an Access token in Code Authorization Grant and read it from the Implicit Grant workflow callback that follows the redirect in step#1



Once you receive an access token, you can use it to access protected resources. Use either the `access_token` parameter, or the Authorization header (recommended) with the Bearer schema and one of the supported grants. TTL for an access token is 24 hours; TTL for a refresh token is 2 weeks since its last usage.

Supported Grants

- **Authorization Code Grant** - requires authorization request and access token request calls.
- **Implicit Grant** - requires an authorization request call.
- **Client Credentials Grant** - requires an access token request call. This grant is available for enterprise accounts only.
- **Refresh Token Grant Type** - requires an access token request call.

Authorization Code Grant

Authorization Code Grant is an OAuth 2.0 flow that enables you to obtain an access token. This grant is used to exchange an authorization code for an access token.

After you return to the client through the redirect URL, the application gets the authorization code from the URL and leverages it to request an access token.

To obtain an access token through the Authorization Code Grant, you must fulfill the following prerequisites and complete the steps thereafter:

1. Prerequisites
2. Obtain an authorization code
3. Obtain an access token

Prerequisites

Data	Description
Client ID	For each application you develop, you must obtain a client identifier key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials Notes: • You must be an enterprise client user or freelancer to request these details. • You must select Key Type as OAuth 2.0 • For more information, see requesting an API key.

	To get the <code>code</code>, access <code>https://upwork.com/ab/messages/rooms/?code=b094053c2892cb819942e2d01e7237e7</code> in by using your Upwork credentials Notes: - You must be an enterprise client user or freelancer to request these details. - You must select Key Type as OAuth 2.0 - For more information, see requesting an API key.
Callback URL	The URI to which Upwork redirects the browser after authentication. This is also known as redirect URI. Notes: - This is applicable to a web project. - For more information, see requesting an API key.

Step 1. Obtaining an authorization code

The Upwork server must request authorization from the resource owner to grant you access to the required data. To do so, you must redirect the resource owner to Upwork server's authentication endpoint.

! Keep the refresh and access token private. If someone obtains your tokens, they could use them to access private user data.

Endpoint

GET <https://www.upwork.com/ab/account-security/oauth2/authorize>

Parameters

response_type	required, string	
		The corresponding grant type. Use code for Authorization Code Grant.
client_id	required, string	
		Your Client ID.
redirect_uri	required, string	
		Redirect URI, must be equal or similar to the callback specified in the key settings.

Returns

Returns the code.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta http-equiv="refresh" content="0;url='https://upwork.com/ab/messages/rooms/?code=b094053c2892cb819942e2d01e7237e7'" />
<title>Redirecting to https://upwork.com/ab/messages/rooms/?code=b094053c2892cb819942e2d01e7237e7</title>
</head>
<body>
Redirecting to <a href="https://upwork.com/ab/messages/rooms/?code=b094053c2892cb819942e2d01e7237e7">https://upwork.com/ab/messages/rooms/?code=b094053c2892cb819942e2d01e7237e7</a>.
</body>
</html>
```

Here, <https://upwork.com/ab/messages/rooms/> is the Callback URL and `b094053c2892cb819942e2d01e7237e7` is the authorization code.

Step 2. Obtaining an access token

! Keep the access token private. If someone obtains your token, they could use it to access private user data.

Parameters

grant_type	required, string
	Grant type. To get a token right after authorization, use authorization_code .
client_id	required, string
	Your Client ID.
client_secret	required, string
	Your Client Secret.
code	required, string
	Authorization code received during the authorization request.
redirect_uri	required, string
	Redirect URI, must be equal or similar to the callback specified in the key settings.

Returns

Returns the access token and its TTL. Example:

```
// Sample request
curl --request POST \
  --url https://api.upwork.com/graphql \
  --header 'Content-Type: application/x-www-form-urlencoded' \
  --header 'Accept: application/json' \
  --data
'client_id=API_KEY_HERE&client_secret=KEY_SECRET_HERE&code=AUTHORIZATION_CODE_FROM_FIRST_STEP&grant_type=authorization_code&redirect_uri=CALLBACK_URL'

// Authorization Code Grant - sample response

{"access_token":"oauth2v2_cec*****60","refresh_token":"oauth2v2_40d*****40","token_type":"Bearer","expires_in":86400}
```

Implicit Grant

Implicit Grant is an OAuth 2.0 flow that enables you to obtain an access token without obtaining an authorization code.

When the system redirects you to the client through the redirect URL, the application appends the access token in the URI. Therefore, unlike the Authorization Code Grant, you do not need to leverage the temporary code to obtain an access token.

To obtain an access token through the Implicit Grant, you must fulfill the following prerequisites and complete the steps thereafter:

1. [Prerequisites](#)
2. [Obtain an access token](#)

Prerequisites

Data	Description
Client ID	For each application you develop, you must obtain a client identifier key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials. Notes: - You must be an enterprise client user or freelancer to request these details. - You must select Key Type as OAuth 2.0 - For more information, see requesting an API key.
Callback URL	The URI to which Upwork redirects the browser after authentication. This is also known as redirect URI. Notes:

Step 1. Obtaining an access token

The Upwork server must request authorization from the resource owner to grant you access to the required data. To do so, you must redirect the resource owner to Upwork server's authentication endpoint.

! Keep the access token private. If someone obtains your token, they could use it to access private user data.

Endpoint

GET <https://www.upwork.com/ab/account-security/oauth2/authorize>

Parameters

response_type	required, string
	The corresponding grant type. Use token for Implicit Grant.
client_id	required, string
	Your Client ID.
redirect_uri	required, string
	Redirect URI, must be equal or similar to the callback specified in the key settings.

Returns

Returns the access token and its TTL. Example:

```
/#access_token=abcdefghijklmnopqrstuvwxyz&expires_in=86399 // Implicit Grant
```

Client Credentials Grant

Client Credentials Grant is an OAuth 2.0 flow that enables you to obtain an access token in server-to-server and client-to-server scenarios. Although users are involved in this flow, you can use the access token outside the context of a user. For example, you can leverage the access token to access the client's resources.

To obtain an access token through the Client Credentials Grant, you must fulfill the following prerequisites and complete the steps thereafter:

1. [Prerequisites](#)
2. [Obtain an access token](#)

! Client Credentials grant is only available to Enterprise consumers

Prerequisites

Data	Description
Client ID	For each application you develop, you must obtain a client identifier key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials. Notes: • You must be an enterprise client user or freelancer to request these details. • You must select Key Type as OAuth 2.0 • For more information, see requesting an API key.
Client Secret	For each application you develop, you must obtain a client shared-secret key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials. Notes:

- For more information, see [requesting an API key](#).

Step 1. Obtaining an access token

! Keep the access token private. If someone obtains your token, they could use it to access private user data.

Endpoint

POST <https://www.upwork.com/api/v3/oauth2/token>

Parameters

grant_type	required, string
	Grant type. To get an access token for the client credentials grant, use client_credentials .
client_id	required, string
	Your Client ID.
client_secret	required, string
	Your Client Secret.

Returns

Returns the access token and its TTL. Example:

```
{
  "access_token": "oauth2v2_584006c0ef4f69fd8278c7769da6ff43",
  "token_type": "Bearer",
  "expires_in": 86400
}
```

Refresh Token Grant Type

Refresh Token Grant Type is an OAuth 2.0 flow that enables you to exchange a refresh token for an access token when the access token has expired. Therefore, you can continue to have a valid access token without further interaction with the user.

To obtain an access token through the Refresh Token Grant Type, you must fulfill the following prerequisites and complete the steps thereafter:

1. [Prerequisites](#)
2. [Obtain an access token](#)

Prerequisites

Data	Description
Client ID	For each application you develop, you must obtain a client identifier key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials. Notes: • You must be an enterprise client user or freelancer to request these details. • You must select Key Type as OAuth 2.0 • For more information, see requesting an API key.
Client Secret	For each application you develop, you must obtain a client shared-secret key. To get this value, access https://www.upwork.com/developer/keys/apply and log in by using your Upwork credentials. Notes: • You must be an enterprise client user or freelancer to request these details.

Step 1. Obtaining an access token

! Keep the refresh token private. If someone obtains your token, they could use it to access private user data.

Endpoint

POST <https://www.upwork.com/api/v3/oauth2/token>

Parameters

grant_type	required, string
	Grant type. To refresh an existing token, having a valid refresh token, use refresh_token .
client_id	required, string
	Your Client ID.
client_secret	required, string
	Your Client Secret.
refresh_token	required, string
	A valid refresh token.

Returns

If you authenticated previously and have a working refresh token, this call returns the refreshed tokens and the TTL of the access token.

```
{
  "access_token": "oauth2v2_daedc8c79a4d5f80b88b2ce953772a0f",
  "refresh_token": "oauth2v2_e8ae4feb6b8d10693d8cff420351461a",
  "token_type": "Bearer",
  "expires_in": 86400
}
```

Service accounts

Service accounts are specific to a service or application and are designed to run applications. Service accounts help you communicate with multiple applications, such as web servers and databases. Unlike user accounts, service accounts are used by system services, for example, web servers, mail transport agents, and databases.

Service accounts are useful when you need to fetch information. For example, you can use service accounts to get information about multiple users in a company. However, you must not use service accounts to perform write operations.

! Note that service accounts are available only for enterprise accounts in Upwork.

Prerequisites for using a Service Account in Upwork

- You must [apply for the client credentials key](#).

After Upwork creates the Client Credentials key and associates it with a service account, you can assign appropriate [scopes](#) and [permissions](#) to the service account.

Requesting a Client Credentials key

To apply for the Client Credentials key:

- Go to <https://www.upwork.com/developer/keys/apply> and provide the requested information about your application. For more information, see the [Getting Started](#) section.

API Key details

[← My Keys](#)

 Important! All API Keys are disabled by default. To expedite your keys review process, please provide full, clear and valid information. [Learn more](#) about what is required.

API Documentation

API usage and instructions for the Upwork APIs can be found at:

www.upwork.com/developer/

Title 



Web project



Desktop project



Mobile project

 Callback URL
Project description 

You MUST provide the following information:

- a clear description on what you plan to use our API for
- the purpose of your project
- whether you use the application internally or publicly

API Usage 

Request Client Credentials Key

☒ I want to create a Client Credential key for Service Account 

3. Select the relevant scopes. For more information, see [scopes](#).
4. Save the changes.

Upwork creates the Client Credentials key and displays the values in a pop-up window. In addition, you can view your key at <https://www.upwork.com/developer/keys/>.

Upwork also creates a Service Account and associates the account with the Client Credentials key. Thereafter, if you perform any actions by using the key, the system actually performs such actions on behalf of the Service Account and not your user account.

Assigning Scopes to Service Accounts

Scopes are collections of permissions to access a specific set of APIs. They are usually grouped by domain and provide more control over sharing your account resources with different roles within your organization.

While applying for the Client Credentials key, you can select the relevant Scopes. These Scopes define the APIs that you can access by using the same key. For more information about Scopes, see [What are Scopes](#).

Assigning permissions to Service Accounts

A Service Account is a user entity. By default, Service Accounts do not have any permissions. To access data by using a Service Account, you must have proper permissions associated with the Service Account.

To assign permissions to Service Accounts:

1. Log in to your Service Account.
2. Under Settings > Members & Permissions, locate the Service Account and update permissions according to the available options. You can set both team and company-level permissions depending on your needs. For more information, see [Team Permissions](#).

Settings

My info

Billing & Payments

Password & Security

Teams

Notification Settings

Financial Accounts

Approvals

Members & Permissions

Tax Information

Connected Services

Upwork Enterprise

Members & Permissions

Owner: Nicholas Thompson (Me)

[Invite New User](#)

Active members Initations

 Search by name

[Filters](#)

☐	User Details	Permissions	
☐	 Delores Heller f305100937kcx1ai Team: Bogan and Sons	Hiring: Full Can Chat With: Team	...
☐	 Lorie Yundt f305100412jfm1yx Team: Renner, Treutel and Schoen	Hiring: Full Can Chat With: Team Enterprise: Compliance	...
☐	 Service Account ysu1bgx6uuxcgtfo Team: Renner, Treutel and Schoen		...

Now, you are ready to leverage your service account to call Upwork's APIs and perform specific operations.

Authorization Request

The Upwork server must request authorization from the resource owner to grant you access to the required data. To do so, you must redirect the resource owner to Upwork server's authentication endpoint.

! Keep the refresh and access token private. If someone obtains your tokens, they could use them to access private user data.

Endpoint

GET <https://www.upwork.com/ab/account-security/oauth2/authorize>

Parameters

response_type	required, string	
		The corresponding Grant Type. Use `code` for Authorization Code Grant and `token` for Implicit Grant. Valid values: code, token
client_id	required, string	
		Your Client ID.
redirect_url	required, string	
		The callback URL.

Returns

Returns a code for the Authorization Code Grant or an access token for the Implicit Grant. Example:

```
/#access_token=abcdefghijklmnopqrstuvwxyz&expires_in=86399 // Implicit Grant
```

Access Token Request

00:59

! Keep the refresh and access token private. If someone obtains your tokens, they could use them to access private user data.

Endpoint

POST <https://www.upwork.com/api/v3/oauth2/token>

Parameters

grant_type	required, string	Grant type. To get a token right after authorization, use `authorization_code`. To refresh an existing token, having a valid `refresh token`, use `refresh_token`. To get an access token for the client credentials grant, use `client_credentials`. Valid values: authorization_code, refresh_token, client_credentials
client_id	required, string	Your Client ID.
client_secret	required, string	Your Client Secret.
code	conditionally required, string	Authorization code received during the authorization request. Required for `authorization_code` grant.
redirect_uri	conditionally required, string	Redirect URI, must be equal or similar to the callback specified in the key settings. Required for `authorization_code` grant.
refresh_token	conditionally required, string	A valid refresh token. Required for `refresh_token` grant.

Returns

If the authorization is successful, this call returns the tokens and the TTL of the access token.

Refresh Token Request

The Access Token request should be used with the proper grant type, which is "refresh_token", in order to refresh the token.

! Keep the refresh and access token private. If someone obtains your tokens, they could use them to access private user data.

Returns

If you authenticated previously and have a working refresh token, this call returns the refreshed tokens and the TTL of the access token.

Libraries and Tools

A library is a collection of behavior implementations, written in a specific programming language. Libraries help you make system calls without the need to re-write code over and over again.

i Python
 PHP
 Ruby
 Java (+ Android, Maven, etc.)
 JavaScript/NodeJS
 Perl
 GO

The following Tools are also available:

i [GQL Explorer](#) - the tool allows you test GraphQL requests
[API Center \(Request new API Key\)](#) - allows you request a new application (API Key)
[API Center \(View existing API Keys\)](#) - manages all requested applications (API Keys)
[API Subscriptions Management Tool](#) - manages your GraphQL subscriptions
[Terms of Use](#) - here you can find terms, rules, and conditions of using Upwork API
[Legacy REST API Docs](#) - Documentation for our Legacy REST APIs can be viewed here. Note that these APIs are planned to be sunset and should only be used as a reference

Operations

Common Information

Typical GraphQL services provide access to two types of requests: Queries and Mutations. The last can include more complicated structures in the request, like Arguments, Aliases, Fragments, Directives, and Variables. Queries are used to fetch the data, where Mutations are needed in order to modify server-side data. For more information, please follow Queries and Mutations section at graphql.org.

00:52

Error handling in GraphQL

If the Upwork server fails while processing a GraphQL request, the system sends an error array response to the client. It is important to note that errors in GraphQL are handled differently from how we might be used to in ReST. While ReST endpoints return HTTP status codes in the error response, for example, **400 - Bad Request** in case of malformed syntax, GraphQL always returns a **200 - OK** status code, regardless of whether the operation succeeded or failed. Consequently, understanding how to decode error responses in GraphQL is crucial to troubleshooting potential operation failures.

Decoding error responses

GraphQL error arrays list all the errors that occurred while processing the API request. Additionally, they provide important information that helps you fix the problem or guide you on what to do next. Let's take a closer look at what's returned.

- **Message:** The error message that helps you understand if the operation succeeded or failed. For example, Validation error of type MissingFieldArgument.
- **Locations:** The exact location in the API request query that results in the error. For example, the line and column parameters in the response.
- **Extensions:** The type of error. For example, ValidationError.

Here is a sample code snippet of a GraphQL error response in Upwork:

```
{
  "errors": [
```

```

      "locations": [
        {
          "line": 3,
          "column": 5
        }
      ],
      "extensions": {
        "classification": "ValidationError"
      }
    },
    {
      "message": "Validation error of type SubSelectionRequired: Sub selection required for type UserVerification of field stepUpVerification @ 'user/stepUpVerification'",
      "locations": [
        {
          "line": 3,
          "column": 5
        }
      ],
      "extensions": {
        "classification": "ValidationError"
      }
    }
  ]
}

```

As the generic status code in GraphQL is not intuitive, you must access the body of the error response to further understand the correct status of the operation.

For example, the following API query returned the status code: **200 - OK**. Let's see how the **message**, **locations**, and **extensions** parameters in the body help you understand the status of the API call.

The screenshot shows a GraphQL API client interface. On the left, the 'Query' tab is active, displaying a GraphQL query. On the right, the 'Result' tab is active, showing the response body. The response is a 200 OK status with a 778ms response time. The response body contains two error objects, each with a message, locations, and extensions.

Query:

```

1 {
2   user {
3     companyFees(organizationId: null,
4       input: { amount: "15" }) {
5       amount
6       fullAmount
7       fees {
8         type
9         description
10        calculationDescription
11        rate
12        amount
13      }
14    }
15  }
16 }
17

```

Result:

```

1 {
2   "errors": [
3     {
4       "message": "Validation error of type
5       NullValueForNonNullArgument: null value for non-null field
6       argument organizationId @ 'user/companyFees'",
7       "locations": [
8         {
9           "line": 3,
10          "column": 5
11        }
12      ],
13      "extensions": {
14        "classification": "ValidationError"
15      }
16     },
17     {
18       "message": "Validation error of type WrongType:
19       argument 'organizationId' with value 'NullValue{}' must not
20       be null @ 'user/companyFees'",
21       "locations": [
22         {
23           "line": 3,
24           "column": 17
25         }
26       ],
27       "extensions": {
28         "classification": "ValidationError"
29       }
30     }
31   ]
32 }

```

Exceptions in GraphQL error types

Typically, GraphQL returns the HTTP status code **200** when the API operation fails at the upstream or data provider service layer. For example, if you do not have OAuth2 permissions or scopes to access an endpoint, the system displays the following error response

200 OK 352ms

 CLEAR

```

1  {
2  "errors": [
3    {
4      "message": "The client or authentication token doesn't have enough oauth2 permissions/scopes to
      access: [CurrentOrganization.id, CurrentOrganization.rid, CurrentUser.email, CurrentUser.id,
      CurrentUser.name, CurrentUser.nid, CurrentUser.rid, Query.organization, Query.user]."
5    }
6  ]

```

However, if the API operation fails at the GraphQL layer itself, the system returns the typical 5XX HTTP status codes, such as **500 - Internal Server Error** if the server encounters an unexpected condition that prevents it from fulfilling the request. In these cases, you can rely on the status code to understand if the operation failed or succeeded.

Sample GraphQL API failure scenarios

When you do not have OAuth2 permissions or scopes to access the endpoint

Query request:

```

{
  user {
    id
    nid
    rid
    name
    email
  }
  organization {
    id
    rid
  }
}

```

Error response:

```

{
  "errors": [
    {
      "message": "The client or authentication token doesn't have enough oauth2 permissions/scopes to access:
[CurrentOrganization.id, CurrentOrganization.rid, CurrentUser.email, CurrentUser.id, CurrentUser.name, CurrentUser.nid,
CurrentUser.rid, Query.organization, Query.user]."
    }
  ]
}

```

When you do not specify any value for a required field

Query request:

```

{
  user {
    companyFees(organizationId: null, input: { amount: "15" }) {
      amount
      fullAmount
      fees {
        type
        description
        calculationDescription
        rate
        amount
      }
    }
  }
}

```

Error response:

```
{
  "errors": [
    {
      "message": "Validation error of type NullValueForNonNullArgument: null value for non-null field argument
organizationId @ 'user/companyFees'",
      "locations": [
        {
          "line": 3,
          "column": 5
        }
      ],
      "extensions": {
        "classification": "ValidationError"
      }
    },
    {
      "message": "Validation error of type WrongType: argument 'organizationId' with value 'NullValue{}' must not be null
@ 'user/companyFees'",
      "locations": [
        {
          "line": 3,
          "column": 17
        }
      ],
      "extensions": {
        "classification": "ValidationError"
      }
    }
  ]
}
```

When you do not define a parameter

Query request:

```
{
  user {
    stepUpVerification
  }
}
```

Error response:

```
{
  "errors": [
    {
      "message": "Validation error of type MissingFieldArgument: Missing field argument stepUpVerificationId @
'user/stepUpVerification'",
      "locations": [
        {
          "line": 3,
          "column": 5
        }
      ],
      "extensions": {
        "classification": "ValidationError"
      }
    },
    {
      "message": "Validation error of type SubSelectionRequired: Sub selection required for type UserVerification of field
stepUpVerification @ 'user/stepUpVerification'",
      "locations": [
        {

```

```
    ],  
    "extensions": {  
      "classification": "ValidationError"  
    }  
  }  
]  
}
```

Using GraphQL through Upwork SDK

Currently, Upwork supports GraphQL SDKs in the following languages:

- Java
- PHP
- Python 3
- Node JS
- Ruby
- Go

SDKs return the GraphQL error responses similar to APIs and do not handle them any differently.

Subscriptions in GraphQL

The **Subscription** feature allows clients to subscribe to specific events within the system. Once a subscribed event occurs, the system automatically sends a payload to the client's designated webhook. This payload includes essential details about the event, such as the **entity**, **action**, and **id** associated with the event.

By utilizing this feature, clients can stay updated with real-time changes, ensuring they are always in sync with the events that matter most to their application or business. The subscription process is flexible and can be configured to trigger on various events, such as updates, creations, or other predefined actions.

i Please, take into account that subscriptions feature is available only to clients. Once a subscription is created, it needs to be reviewed and approved by Upwork Team. Until then, the subscription will remain in REVIEW state.

Key Components:

- **Entity:** Represents the object or resource associated with the event (e.g., "JP", "OFFER", "JA").
- **Action:** Describes the type of event or operation performed (e.g., "NEW", "UPDATE", "CANCELLED").
- **ID:** A unique identifier associated with the event, typically referencing a specific record or transaction.

When an event that the client has subscribed to is triggered, the system will send a payload to the client's webhook endpoint. This payload will include the entity name, action taken, and a unique identifier (ID) to allow for easy tracking and handling of the event.

Example Payload:

```
{  
  "entity": "OFFER",  
  "action": "CREATE",  
  "id": "123456789"  
}
```

To manage your subscriptions and configure events easily, you can use our [Subscription Management Tool](#). This tool allows you to view, create, and update your subscriptions through a user-friendly interface.

This ensures that clients receive the necessary data to process events in real-time, giving them complete control over how events are handled within their application. The flexibility to define custom webhooks and subscribe to a range of events provides a powerful way to integrate the system's activities with the client's workflows.

Supported events:

[Job postings \(JP\)](#) [Offer \(OFFER\)](#) [Vendor Job Application \(Vendor JA\)](#) [Client Job Application \(Client JA\)](#) [Milestone \(MILESTONE\)](#)
[Contract Feedback \(CFB\)](#)

Subscription events when any action is initiated by a Client and performed on any marketplace job.

Stack OverflowGetting StartedTOSFAQChangelog			
Job postings (JP)	NEW	new marketplace job posting was created	
	UPDATE	marketplace job posting was updated	
	CLOSE	marketplace job posting was closed	

Queries

Job Posts

jobPosting

Get job posting by its ID

Required Permissions
"Job Postings - Read-Only Access"

Response
Returns a **JobPosting!**

Arguments	
Name	Description
jobPostingId - ID!	Job posting identifier
loadAnnotation - Boolean	Include annotations

QUERY

```
query jobPosting(  
  $jobPostingId: ID!,  
  $loadAnnotation: Boolean  
) {  
  jobPosting(  
    jobPostingId: $jobPostingId,  
    loadAnnotation: $loadAnnotation  
  ) {  
    id  
    version  
    info {  
      ...JobPostingInfoFragment  
    }  
    visibility  
    ownership {  
      ...JobPostingOwnershipFragment  
    }  
    content {  
      ...JobPostingContentFragment  
    }  
    attachment {  
      ...JobPostingAttachmentFragment  
    }  
    classification {  
      ...JobPostingClassificationFragment  
    }  
    segmentationData {  
      ...JobPostingSegmentationDataFragment  
    }  
    contractTerms {
```

```
...JobPostingContractorSelectionFragment
}
additionalInfo {
...JobPostingAdditionalInfoFragment
}
ptclInfo {
...JobPostingPtclInfoFragment
}
proposalsStatistics {
...ProposalsStatisticsFragment
}
customFields {
...JobPostingCustomFieldsFragment
}
}
}
```

VARIABLES

```
{
  "jobPostingId": "4",
  "loadAnnotation": false
}
```

RESPONSE

```
{
  "data": {
    "jobPosting": {
      "id": "4",
      "version": 123,
      "info": JobPostingInfo,
      "visibility": "PUBLIC_INDEX",
      "ownership": JobPostingOwnership,
      "content": JobPostingContent,
      "attachment": [JobPostingAttachment],
      "classification": JobPostingClassification,
      "segmentationData": JobPostingSegmentationData,
      "contractTerms": JobPostingContractTerms,
      "contractorSelection": JobPostingContractorSelection,
      "additionalInfo": JobPostingAdditionalInfo,
      "ptclInfo": JobPostingPtclInfo,
      "proposalsStatistics": ProposalsStatistics,
      "customFields": [JobPostingCustomFields]
    }
  }
}
```

marketplaceJobPosting

Get marketplace job posting by its ID

Required Permissions
"Read marketplace Job Postings"

Response
Returns a MarketplaceJobPosting

Arguments

Name	Description
id - ID!	Job posting identifier

```
query marketplaceJobPosting($id: ID!) {
  marketplaceJobPosting(id: $id) {
    id
    workflowState {
      ...MarketplacePostingWorkFlowStateFragment
    }
    ownership {
      ...MarketplacePostingOwnershipFragment
    }
    annotations {
      ...MarketplacePostingAnnotationsFragment
    }
    activityStat {
      ...MarketplacePostingActivityStatFragment
    }
    content {
      ...MarketplaceJobPostingContentFragment
    }
    attachments {
      ...MarketplacePostingAttachmentFragment
    }
    classification {
      ...MarketplacePostingClassificationFragment
    }
    segmentationData {
      ...MarketplaceSegmentationDataFragment
    }
    contractTerms {
      ...MarketplaceContractTermsFragment
    }
    contractorSelection {
      ...MarketplaceContractorSelectionFragment
    }
    additionalSearchInfo {
      ...MarketplaceAdditionalSearchInfoFragment
    }
    clientCompany {
      ...PrivateCompanyInfoFragment
    }
    clientCompanyPublic {
      ...MarketplacePublicCompanyInfoFragment
    }
    canClientReceiveContractProposal
    clientProposals {
      ...ClientProposalsConnectionFragment
    }
    customFields {
      ...CustomFieldsConnectionFragment
    }
  }
}
```

VARIABLES

```
{ "id": 4 }
```

RESPONSE

```
{
  "data": {
    "marketplaceJobPosting": {
      "id": "4",
      "workflowState": MarketplacePostingWorkFlowState,
      "ownership": MarketplacePostingOwnership,
      "annotations": MarketplacePostingAnnotations,
      "activityStat": MarketplacePostingActivityStat,
      "content": MarketplaceJobPostingContent,
      "attachments": [MarketplacePostingAttachment],
```

```
"contractorSelection": MarketplaceContractorSelection,
"additionalSearchInfo": MarketplaceAdditionalSearchInfo,
"clientCompany": PrivateCompanyInfo,
"clientCompanyPublic": MarketplacePublicCompanyInfo,
"canClientReceiveContractProposal": false,
"clientProposals": ClientProposalsConnection,
"customFields": CustomFieldsConnection
}
}
}
```

marketplaceJobPostings

Deprecated Will be removed in future release. Please Use marketplaceJobPostingsSearch instead

Required Permissions

"Read marketplace Job Postings"

Response

Returns a [MarketplaceJobPostingSearchConnection](#)

Arguments

Name	Description
marketPlaceJobFilter - MarketplaceJobFilter	Filter to search job postings
searchType - MarketplaceJobPostingSearchType	Search type context
sortAttributes - [MarketplaceJobPostingSearchSortAttribute]	Sort attributes

QUERY

```
query marketplaceJobPostings(
  $marketPlaceJobFilter: MarketplaceJobFilter,
  $searchType: MarketplaceJobPostingSearchType,
  $sortAttributes: [MarketplaceJobPostingSearchSortAttribute]
){
  marketplaceJobPostings(
    marketPlaceJobFilter: $marketPlaceJobFilter,
    searchType: $searchType,
    sortAttributes: $sortAttributes
  ){
    totalCount
    edges {
      ...MarketplaceJobpostingSearchEdgeFragment
    }
    pageInfo {
      ...PageInfoFragment
    }
  }
}
```

VARIABLES

```
{
  "marketPlaceJobFilter": MarketplaceJobFilter,
  "searchType": "USER_JOBS_SEARCH",
  "sortAttributes": [
    MarketplaceJobPostingSearchSortAttribute
  ]
}
```

```
{
  "data": {
    "marketplaceJobPostings": {
      "totalCount": 987,
      "edges": [MarketplaceJobpostingSearchEdge],
      "pageInfo": PageInfo
    }
  }
}
```

marketplaceJobPostingsContents

Get marketplace job postings content by ID

Required Permissions

"Read marketplace Job Postings"

Response

Returns [MarketplaceJobPostingsContents]!

Arguments

Name	Description
ids - [ID!]!	Job posting identifiers

QUERY

```
query marketplaceJobPostingsContents($ids: [ID!]!) {
  marketplaceJobPostingsContents(ids: $ids) {
    id
    ciphertext
    title
    description
    publishedDateTime
    annotations {
      ...MarketplacePostingAnnotationsFragment
    }
  }
}
```

VARIABLES

```
{
  "ids": [4]
}
```

RESPONSE

```
{
  "data": {
    "marketplaceJobPostingsContents": [
      {
        "id": "4",
        "ciphertext": "xyz789",
        "title": "abc123",
        "description": "xyz789",
        "publishedDateTime": "abc123",
        "annotations": MarketplacePostingAnnotations
      }
    ]
  }
}
```


countries

List of countries. Currently no filtering is available

Required Permissions

"Common Functionality - Read And Write Access"

Response

Returns [\[Country!\]](#)

QUERY

```
query countries {  
  countries {  
    id  
    name  
    twoLetterAbbreviation  
    threeLetterAbbreviation  
    region  
    phoneCode  
    relatedRegion {  
      ...RegionFragment  
    }  
    relatedSubRegion {  
      ...RegionFragment  
    }  
    active  
    registrationAllowed  
  }  
}
```

RESPONSE

```
{  
  "data": {  
    "countries": [  
      {  
        "id": 4,  
        "name": "abc123",  
        "twoLetterAbbreviation": "abc123",  
        "threeLetterAbbreviation": "abc123",  
        "region": "EU",  
        "phoneCode": "abc123",  
        "relatedRegion": Region,  
        "relatedSubRegion": Region,  
        "active": true,  
        "registrationAllowed": true  
      }  
    ]  
  }  
}
```

languages

Get the list of languages

Required Permissions

One of: "Common Entities - Read-Only Access" OR "Common Functionality - Read And Write Access"

QUERY

```
query languages {
  languages {
    iso639Code
    active
    englishName
  }
}
```

RESPONSE

```
{
  "data": {
    "languages": [
      {
        "iso639Code": "abc123",
        "active": true,
        "englishName": "abc123"
      }
    ]
  }
}
```

reasons

Reasons for some actions. Examples - declining invitation, ending contract, withdrawing offer

Required Permissions

"Common Functionality - Read And Write Access"

Response

Returns `[ReasonsMetadata]!`

Arguments

Name	Description
reasonType - <code>ReasonType!</code>	Reason category type
all - <code>Boolean</code>	Return all reasons (ignore visibility filters)

QUERY

```
query reasons(
  $reasonType: ReasonType!,
  $all: Boolean
){
  reasons(
    reasonType: $reasonType,
    all: $all
  ){
    id
    reason
    alias
  }
}
```

VARIABLES

RESPONSE

```
{
  "data": {
    "reasons": [
      {
        "id": 4,
        "reason": "xyz789",
        "alias": "abc123"
      }
    ]
  }
}
```

regions

Get the list of regions

Required Permissions

"Common Functionality - Read And Write Access"

Response

Returns [\[Region!\]](#)

QUERY

```
query regions {
  regions {
    id
    name
    parentRegion {
      ...RegionFragment
    }
  }
}
```

RESPONSE

```
{
  "data": {
    "regions": [
      {
        "id": 4,
        "name": "xyz789",
        "parentRegion": Region
      }
    ]
  }
}
```

requestMetadata

Information about request - whether it is an internal requests, in specific mode, etc

Required Permissions

"Common Functionality - Read And Write Access"

QUERY

```
query requestMetadata {  
  requestMetadata {  
    sudo  
    internal  
  }  
}
```

RESPONSE

```
{"data": {"requestMetadata": {"sudo": true, "internal": false}}}
```

timeZones

List of all available time zones

Required Permissions

"Common Functionality - Read And Write Access"

Response

Returns [TimeZoneRecord!]

QUERY

```
query timeZones {  
  timeZones {  
    timeZoneName  
    timeZoneDescription  
  }  
}
```

RESPONSE

```
{  
  "data": {  
    "timeZones": [  
      {  
        "timeZoneName": "abc123",  
        "timeZoneDescription": "abc123"  
      }  
    ]  
  }  
}
```

visitor

Visitor details

Required Permissions

"Common Functionality - Read And Write Access"

Response

QUERY

```
query visitor {
  visitor {
    i18n {
      ...VisitorI18NFragment
    }
    segment {
      ...VisitorSegmentFragment
    }
  }
}
```

RESPONSE

```
{
  "data": {
    "visitor": {
      "i18n": VisitorI18N,
      "segment": VisitorSegment
    }
  }
}
```

Offers and Contracts

contract

Deprecated Will be removed in future release. Use contractByTerm with modernised ContractDetails schema

Fetch contract information by ID. Note: The provided 'id' corresponds to the id of the contract term in the ContractDetails schema.

Required Permissions
"Offer - Read-Only Access"

Response
Returns a **Contract**

Arguments

Name	Description
id - ID!	Contract identifier

QUERY

```
query contract($id: ID!) {
  contract(id: $id) {
    id
    termId
    title
    contractType
    job {
      ...MarketplaceJobPostingFragment
    }
    hasAgency
    offer {
      ...OfferFragment
    }
  }
}
```

```
vendorProposal {
  ...VendorProposalFragment
}
weeklyChargeAmount {
  ...MoneyFragment
}
weeklyHoursLimit
hourlyChargeRate {
  ...MoneyFragment
}
createdDateTime
firstContractStartDateTime
modifiedDateTime
startDateTime
endDateTime
manualTimeAllowed
hideScreenshots
last
weeklyStipendHours
estimatedDuration {
  ...ProposalEngagementDurationFragment
}
status
suspended
freelancer {
  ...FreelancerProfileFragment
}
paused
clientCompany {
  ...PublicCompanyInfoFragment
}
milestones {
  ...MilestoneFragment
}
contractId
workDays {
  ...WorkDayFragment
}
workDiaryTimeCells {
  ...WorkDiaryTimeCellFragment
}
feedback {
  ...FeedbackFragment
}
}
```

VARIABLES

```
{ "id": "4" }
```

RESPONSE

```
{
  "data": {
    "contract": {
      "id": 4,
      "termId": "4",
      "title": "xyz789",
      "contractType": "HOURLY",
      "job": MarketplaceJobPosting,
      "hasAgency": true,
      "offer": Offer,
      "clientProposal": ClientProposal,
      "vendorProposal": VendorProposal,
      "weeklyChargeAmount": Money,
      "weeklyHoursLimit": 987.65,
      "hourlyChargeRate": Money,
      "createdDateTime": "abc123",
```